

FORENSIK DIGITAL

UAS

KELAS IF-03-02

Dosen: FND



Anggota Kelompok:

Arka Dwi Indrastata – 1203230017

PROGRAM STUDI INFORMATIKA

FAKULTAS INFORMATIKA

UNIVERSITAS TELKOM

SURABAYA

2026

1.LAPORAN 1

Keterangan User

AKUN	NAMA
USER A	KORBAN (EMULATOR)
USER B	TEMAN 1 (Haidar)
USER C	TEMAN 2 (Meraa)
USER D	TEMAN 3 (Ryan)

Aktivitas Pengguna

NO	WAKTU	AKTIFITAS
1.	22:39	User A melakukan Login ke Discord
2.	22:40	User A mengirim pesan ke User C (MERAA)
3.	22:41	User A mengirim pesan ke User B (HAIDAR)
4.	22:42	User A mengirim pesan ke User D (RYAN)
5.	22:42	User B membalas pesan User A
6.	22:40	User C membalas pesan User A
7.	22:45	User D membalas pesan User A
8.	22:47	User A mengirim emoji ke User B (HAIDAR)
9.	22:49	User A mengirim gambar ke User D (RYAN)
10.	22:49	User A mengirim video ke User D (HAIDAR)
11.	22:52	User A mengirim mengedit pesan ke User B (HAIDAR)
12.	22:52	User A menghapus pesan ke User C (MERAA)
13.	22:53	User A mengarsipkan chat User B (HAIDAR)
14.	22:54	User A melakukan voice call kepada User B dan gagal (HAIDAR)
15.	22:55	User B menelepon User A
16.	22:56	User A melakukan voice call kepada User D (RYAN)
17.	22:58	User A membuat grup yang Bernama " forensic "

		grub” yang beranggotakan User B dan User D (HAIDAR dan RYAN)
18.	22:58	User A melakukan chat di grub
19.	22:58	User D membalas chat di grub
20.	22:59	User A keluar dari Telegram

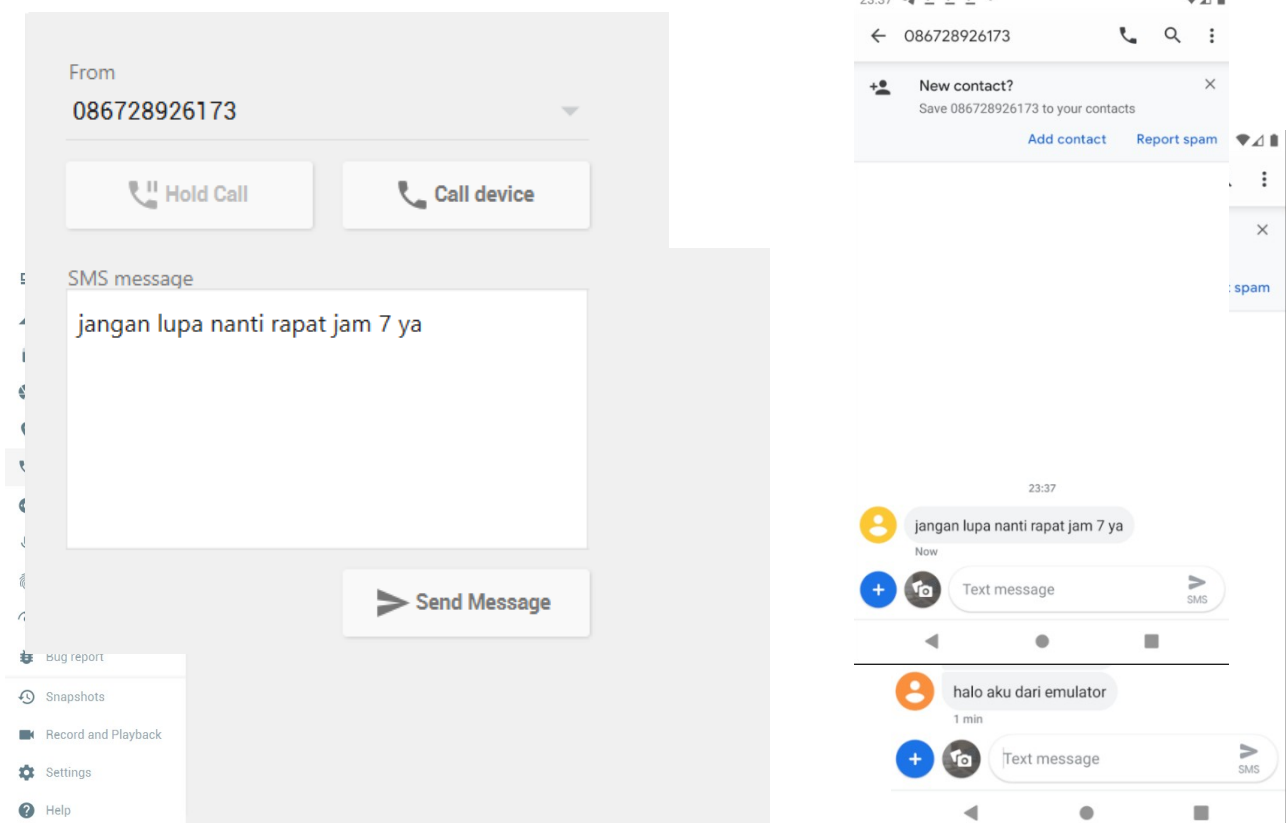
2.A. SIMULASI SMS

Langkah 1 - Membuka Emulator

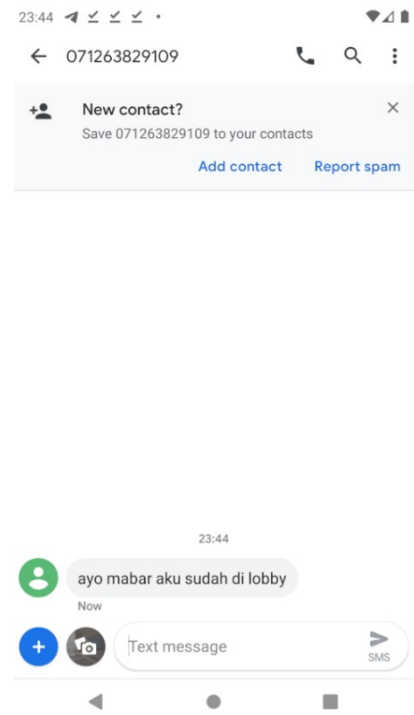
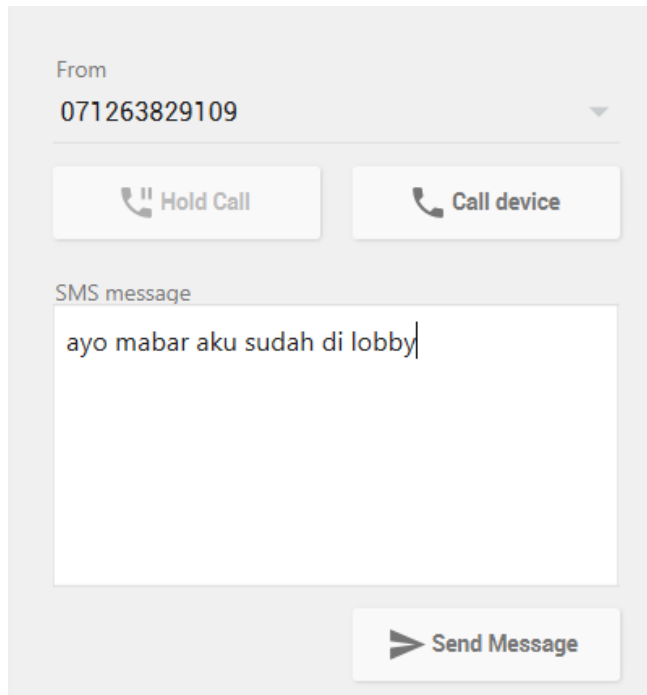
Jalankan emulator lalu buka bagian titik tidak di kanan layar, pilih phone

Langkah 2 – Simulasi SMS Masuk 1

Langkah 3 – Simulasi SMS Masuk 2



Langkah 4 – Simulasi SMS Masuk 2



Langkah 5 – Hapus Salah Satu SMS

Masuk ke aplikasi pesan dan hapus salah satu pesan

SEBELUM DIHAPUS DIHAPUS



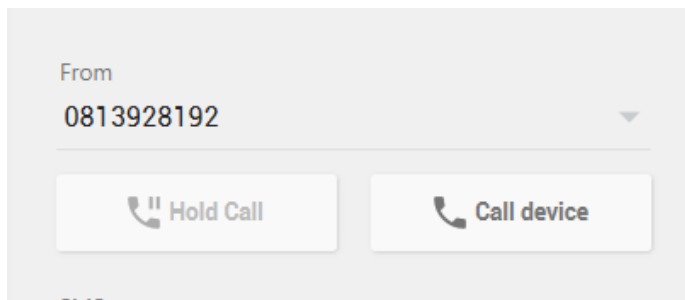
SESUDAH



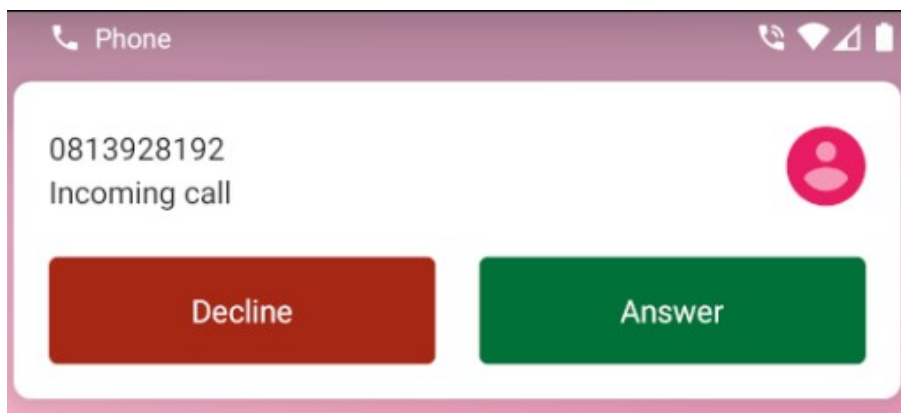
2.B. SIMULASI PANGGILAN TELEPON

Panggilan Masuk

Masuk di bagian menu extended control lagi tapi kali ini alih alih mengirim pesan, kita akan mengirim sebuah panggilan



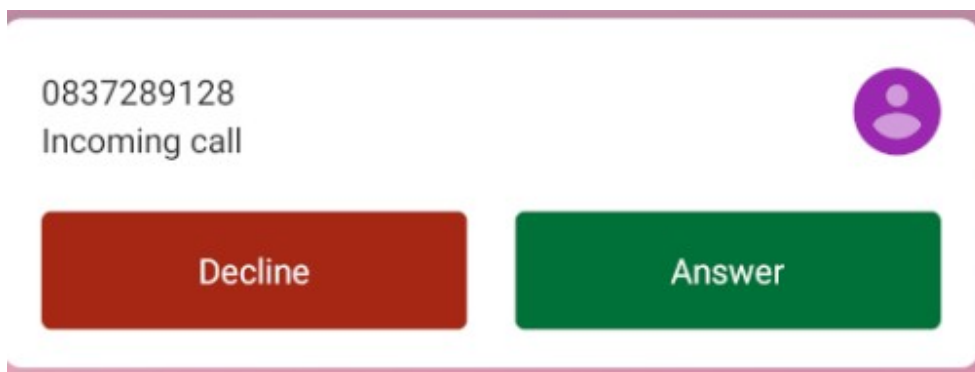
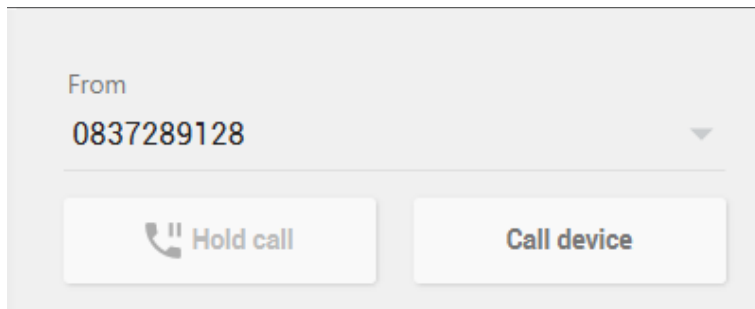
Setelah memasukkan nomor asal maka pilih menu call device, nantinya akan ada telepon yang masuk di emulator



Jawab panggilanya lalu biarkan selama kurang lebih 30 detik

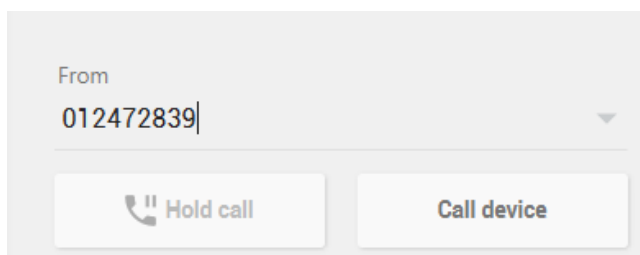
Miss Call

Sama seperti tadi bedanya disini jangan diangkat telepon nya biarkan berdering hingga mati sendiri



Incoming Call Kedua

Angkat lalu biarkan selama 10 detik lalu putuskan



Kesimpulan

Simulasi aktivitas SMS dan telepon dilakukan menggunakan fitur Phone pada Android Emulator (Extended Controls). Metode ini digunakan karena emulator tidak menggunakan SIM Card fisik, namun Android Emulator menyediakan fasilitas simulasi SMS dan

panggilan telepon yang menghasilkan artefak digital pada sistem Android sebagaimana perangkat nyata.

3.IDENTIFIKASI BARANG BUKTI

Langkah 1 - Verifikasi Koneksi Emulator

Buka powershell lalu jalankan **“./adb devices”**

```
PS D:\Kuliah\Semester_6\Forensik digital\UAS\ADB> ./adb devices
List of devices attached
emulator-5554    device
```

Jika ada tulisan emulator maka kita sudah berhasil connect ke emulator

Langkah 2 – Identifikasi Devices

Gunakan perintah **“./adb shell getprop ro.product.model”**, untuk melihat model emulator yang kita gunakan

```
PS D:\Kuliah\Semester_6\Forensik digital\UAS\ADB> ./adb shell getprop ro.product.model
Android SDK built for x86
```

Disini tertulis x86, yang berarti kita menggunakan arsitektur x86

Langkah 3 – Identifikasi Versi Android

Gunakan perintah **“./adb shell getprop ro.build.version.release”**, untuk melihat versi android yang kita gunakan

```
PS D:\Kuliah\Semester_6\Forensik digital\UAS\ADB> ./adb shell getprop ro.build.version.release
10
```

Disini emulator kita menggunakan android versi 10

Langkah 4 – Identifikasi Nama Package Telegram

Gunakan perintah **“./ adb shell pm list packages | findstr telegram”**, untuk melihat atau mencari package telegram yang nantinya akan digunakan untuk analisis

```
PS D:\Kuliah\Semester_6\Forensik digital\UAS\ADB> ./adb shell pm list packages | findstr telegram
package:org.telegram.messenger.web
```

Disini nama package telegramnya yaitu
org.telegram.messenger.web

Kesimpulan

Setelah seluruh aktivitas pengguna selesai disimulasikan, dilakukan proses identifikasi terhadap perangkat yang akan dijadikan barang bukti digital. Tahap ini bertujuan untuk mendokumentasikan karakteristik perangkat sebelum dilakukan proses akuisisi data.

Perangkat yang digunakan dalam penelitian ini berupa Android Emulator pada Android Studio dengan spesifikasi sebagai berikut:

Tabel Identifikasi Barang Bukti :

PARAMETER	NILAI
Jenis Perangkat	Android Emulator
Model Perangkat	Android SDK built for x86
Sistem Operasi	Android 10
Aplikasi Target	Telegram
Package Name	org.telegram.messenger.web

Hasil identifikasi menunjukkan bahwa perangkat berhasil terhubung melalui ADB dan dapat diakses untuk keperluan investigasi forensik. Aplikasi Telegram berhasil teridentifikasi dengan package name

org.telegram.messenger.web yang selanjutnya akan menjadi target utama proses akuisisi dan analisis artefak digital.

4. VERIFIKASI AKSES FORENSIK

Setelah identifikasi perangkat selesai dilakukan, tahap berikutnya adalah verifikasi tingkat akses terhadap sistem Android. Tujuan tahap ini adalah untuk menentukan metode akuisisi yang dapat digunakan selama investigasi.

Verifikasi dilakukan menggunakan Android Debug Bridge (ADB) dengan menjalankan perintah `adb root` dan `whoami`.

Hasil pengujian menunjukkan bahwa Android Debug Daemon (adbd) berjalan dengan hak akses root. Verifikasi lanjutan menggunakan perintah `whoami` menghasilkan nilai root yang menandakan investigator memiliki akses penuh terhadap sistem file Android.

```
PS D:\Kuliah\Semester_6\Forensik digital\UAS\ADB> ./adb root
adbd is already running as root
PS D:\Kuliah\Semester_6\Forensik digital\UAS\ADB> ./adb shell
generic_x86:/ # whoami
root
```

Berdasarkan hasil tersebut, perangkat dapat dilakukan proses logical acquisition maupun physical acquisition karena investigator memiliki hak akses penuh terhadap sistem operasi Android termasuk direktori aplikasi yang berada pada partisi data.

5.A. LOGICAL ACQUISITION INFORMASI PERANGKAT

Langkah 1 – Akuisisi Informasi Sistem

Jalankan perintah **`./adb shell getprop > "..\Logical_Acquisitoon\Device_Info\getprop.txt"`**, perintah ini nantinya akan membuat sebuah file getprop.txt ke path yang sudah ditentukan.

Name	Date modified	Type	Size
 getprop.txt	23/06/2026 01:17	Text Document	36 KB

File ini berisi ratusan properti Android seperti:

ro.build.version.release

ro.product.model

ro.build.fingerprint

ro.hardware

ro.kernel.qemu

Dari sini nanti kita nanti bisa membuktikan Versi android, Tipe emulator, Build Android, Informasi sistem lainnya.

Langkah 2 – Akuisisi Daftar Package

Sama seperti tadi, Jalankan perintah **`./adb shell pm list packages > "..\Logical_Acquisition\Device_Info\packages.txt"`** di powershell, perintah ini nantinya akan membuat sebuah file packages.txt ke path yang sudah ditentukan.

Name	Date modified	Type	Size
 getprop.txt	23/06/2026 01:17	Text Document	36 KB
 packages.txt	23/06/2026 01:20	Text Document	15 KB

File ini berisi seluruh aplikasi yang terinstal saat investigasi dilakukan.

package:org.telegram.messenger.web

package:com.android.messaging

package:com.android.dialer

Nanti saat analisis Telegram, SMS, dan Call Log, kita bisa membuktikan bahwa aplikasi tersebut memang ada pada perangkat.

Langkah 3 – Dokumentasi Kondisi Perangkat

Jalankan perintah ini di powershell **“./adb exec-out screencap -p > “..\\Logical_Acquisition\\Screenshots\\homescreen.png”**. Dalam forensik digital, screenshot awal berfungsi seperti foto TKP.

Screenshot menunjukkan:

- kondisi perangkat saat diamankan
- aplikasi yang terlihat
- status perangkat saat akuisisi dimulai

Kesimpulan

Tahap awal Logical Acquisition dilakukan dengan mengumpulkan informasi dasar perangkat menggunakan Android Debug Bridge (ADB). Akuisisi ini bertujuan untuk memperoleh identitas perangkat serta mendokumentasikan kondisi perangkat sebelum dilakukan ekstraksi data lebih lanjut.

Akuisisi dilakukan terhadap tiga jenis artefak, yaitu informasi sistem Android, daftar aplikasi yang terpasang, dan dokumentasi visual kondisi perangkat.

NO	Artefak	Deskripsi
1.	Getprop.txt	Informasi sistem Android dan properti

		perangkat
2.	packages.txt	Daftar aplikasi yang terpasang pada perangkat
3.	homescreen.png	Screenshot kondisi perangkat saat proses akuisisi

Informasi sistem digunakan untuk mengidentifikasi karakteristik perangkat seperti versi Android, model perangkat, dan konfigurasi sistem. Daftar aplikasi digunakan untuk memverifikasi keberadaan aplikasi target investigasi, yaitu Telegram. Dokumentasi visual perangkat digunakan untuk merekam kondisi perangkat pada saat proses akuisisi dimulai.

5.B. LOGICAL ACQUISITION

AKUISISI ARTEFAK SMS DAN CALL LOG

Langkah 1 – Akuisisi SMS

Jalankan perintah berikut ini di powershell **“./adb shell content query --uri content://sms”**.Nantinya akan tertampil semua sms yang tersedia, mulai dari nomor pengirim, timestamp, isi pesan, status pesan.

```
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb shell content query --uri content://sms
Row: 0 _id=4, thread_id=5, address=071263829109, person=NULL, date=1782146649627, date_sent=1782146650000, protocol=0, read=1, status=-1, type=1, reply_path_present=0, subject=NULL, body=ayo mabar aku sudah di lobby, service_center=NULL, locked=0, sub_id=1, error_code=0, creator=com.google.android.apps.messaging, seen=1
Row: 1 _id=2, thread_id=3, address=08121709038, person=NULL, date=1782146070927, date_sent=1782146071000, protocol=0, read=1, status=-1, type=1, reply_path_present=0, subject=NULL, body=halo aku dari emulator, service_center=NULL, locked=0, sub_id=1, error_code=0, creator=com.google.android.apps.messaging, seen=1
Row: 2 _id=1, thread_id=3, address=08121709038, person=NULL, date=1782146068974, date_sent=1782146067000, protocol=0, read=1, status=-1, type=1, reply_path_present=0, subject=NULL, body=halo aku dari emulator, service_center=NULL, locked=0, sub_id=1, error_code=0, creator=com.google.android.apps.messaging, seen=1
```

Karena sudah bisa berhasil menampilkan data, maka jalankan perintah selanjutnya yaitu **“./adb shell content query --uri content://sms > “..\Logical_Acquisition\SMS\sms_content.txt”**.

Perintah ini akan menyimpan data tadi ke dalam local pc kita dengan format .txt

Name	Date modified	Type	Size
 sms_content.txt	23/06/2026 15:21	Text Document	2 KB

Langkah 2 - Akuisisi Call Log

Sama seperti Langkah 1 tadi, kita akan melihat data dari call log nya terlebih dahulu sebelum menyimpan di local pc, menggunakan perintah **`./adb shell content query --uri content://call_log/calls`**.

```
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb shell content query --uri content://call_log/calls
Row: 0 formatted_number=NULL, duration=31, subscription_id=8901410321118510720, photo_id=0, post_dial_digits=, call_scre
eening_app_name=NULL, number=0813928192, countryiso=US, photo_uri=NULL, geocoded_location=, block_reason=0, subscription
_component_name=com.android.phone/com.android.services.telephony.TelephonyConnectionService, add_for_all_users=1, number
type=NULL, features=0, transcription=NULL, last_modified=1782147914868, _id=1, new=1, date=1782147871661, name=NULL, typ
e=1, presentation=1, via_number=, numberlabel=NULL, normalized_number=NULL, phone_account_address=+15555215554, lookup_u
ri=NULL, voicemail_uri=NULL, matched_number=NULL, transcription_state=0, data_usage=NULL, call_screening_component_name=
NULL, is_read=NULL
Row: 1 formatted_number=NULL, duration=5, subscription_id=8901410321118510720, photo_id=0, post_dial_digits=, call_scre
eening_app_name=NULL, number=0837289128, countryiso=US, photo_uri=NULL, geocoded_location=, block_reason=0, subscription
_component_name=com.android.phone/com.android.services.telephony.TelephonyConnectionService, add_for_all_users=1, numberr
type=NULL, features=0, transcription=NULL, last_modified=1782148116215, _id=2, new=1, date=1782147985145, name=NULL, type
=1, presentation=1, via_number=, numberlabel=NULL, normalized_number=NULL, phone_account_address=+15555215554, lookup_ur
i=NULL, voicemail_uri=NULL, matched_number=NULL, transcription_state=0, data_usage=NULL, call_screening_component_name=N
ULL, is_read=NULL
Row: 2 formatted_number=NULL, duration=11, subscription_id=8901410321118510720, photo_id=0, post_dial_digits=, call_scre
eening_app_name=NULL, number=012472839, countryiso=US, photo_uri=NULL, geocoded_location=, block_reason=0, subscription
_component_name=com.android.phone/com.android.services.telephony.TelephonyConnectionService, add_for_all_users=1, numberr
type=NULL, features=0, transcription=NULL, last_modified=1782148352190, _id=3, new=1, date=1782148309033, name=NULL, type
=1, presentation=1, via_number=, numberlabel=NULL, normalized_number=NULL, phone_account_address=+15555215554, lookup_ur
i=NULL, voicemail_uri=NULL, matched_number=NULL, transcription_state=0, data_usage=NULL, call_screening_component_name=N
ULL, is_read=NULL
```

Disini kita bisa melihat nomor telepon pengirim, tanggal, durasi. Setelah mendapatkan data tersebut, seperti di Langkah 1 kita akan mengimport data tersebut ke dalam lokal pc kita dengan menggunakan perintah **`./adb shell content query --uri content://call_log/calls > "..\Logical_Acquisition\CallLogs\callog_content.txt"`**.

Name	Date modified	Type	Size
 callog_content.txt	23/06/2026 15:35	Text Document	5 KB

Dokumentasi Tambahan

Disini kami ingin melakukan screenshot tampilan sms dan call log untuk menguatkan bukti forensic. Untuk sms kami menggunakan perintah:

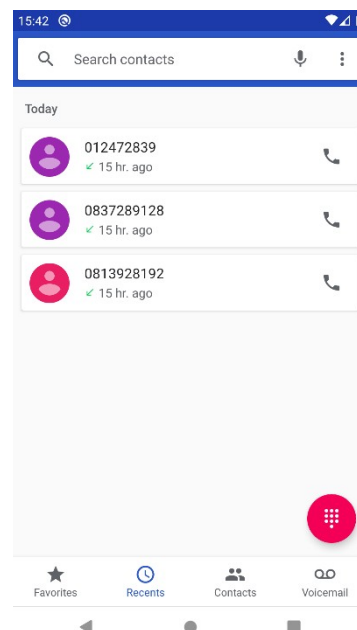
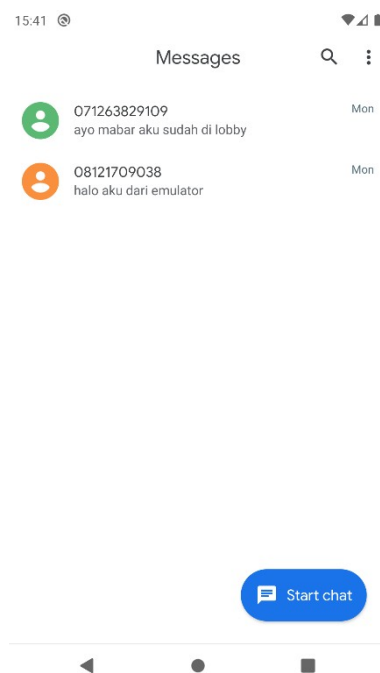
```
./adb shell screencap -p /sdcard/sms.png
```

```
./adb pull /sdcard/sms.png "..\Logical_Acquisition\Screenshots\ sms.png"
```

Sedangkan untuk call log, kami menggunakan perintah

```
./adb shell screencap -p /sdcard/callog.png
```

```
./adb pull /sdcard/callog.png "..\Logical_Acquisition\ Screenshots\callog.png"
```



Kesimpulan

Tahap berikutnya dalam proses Logical Acquisition adalah pengambilan artefak SMS dan Call Log yang tersimpan pada perangkat Android. Akuisisi dilakukan menggunakan Android Content Provider melalui Android Debug Bridge (ADB).

Metode ini dipilih karena memungkinkan investigator memperoleh data komunikasi yang tersimpan pada sistem Android tanpa melakukan modifikasi terhadap data asli.

Artefak yang diperoleh meliputi:

NO	Artefak	Deskripsi
1.	Sms_content.txt	Data SMS yang tersimpan pada perangkat
2.	Callog_content.txt	Data riwayat panggilan telepon
3.	sms.png	Dokumentasi visual aplikasi SMS
4.	Callog.png	Dokumentasi visual riwayat panggilan

Data SMS digunakan untuk mengidentifikasi nomor pengirim, isi pesan, dan waktu komunikasi. Data Call Log digunakan untuk mengidentifikasi nomor telepon, jenis panggilan, durasi panggilan, serta waktu terjadinya komunikasi. Seluruh artefak disimpan sebagai bukti digital untuk proses analisis dan rekonstruksi timeline pada tahap berikutnya.

6.DISCOVERY ARTEFAK TELEGRAM

Sebelum kami melakukan akuisisi, kami akan terlebih dahulu mencari data telegram disimpan Dimana dengan menggunakan perintah berikut ini :

Pertama tama kami akan masuk ke root dengan menggunakan perintah **"./adb root"**. Setelah itu masuk ke shell dengan perintah **"./adb shell"**, jika sudah masuk ke dalam shell baru berikan perintah untuk mencari data di telegram, berikut adalah perintahnya **" ls -la /data/data/ | grep telegram"**.

```
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb root
adb is already running as root
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb shell
generic_x86:/ # ls -la data/data/ | grep telegram
drwx----- 10 u0_a146      u0_a146      4096 2026-06-23 15:12 org.telegram.messenger.web
generic_x86:/ # |
```

Setelah aplikasi target berhasil diidentifikasi, dilakukan proses discovery untuk menemukan lokasi penyimpanan data aplikasi Telegram pada sistem Android. Proses discovery dilakukan menggunakan Android Debug Bridge (ADB) dengan hak akses root. Hasil pemeriksaan menunjukkan bahwa direktori data aplikasi Telegram berhasil ditemukan pada lokasi:

/data/data/org.telegram.messenger.web

Direktori tersebut merupakan lokasi penyimpanan utama aplikasi Telegram yang berisi berbagai artefak digital seperti database, file konfigurasi, cache aplikasi, serta data pengguna lainnya.

Temuan ini menjadi dasar untuk proses akuisisi artefak Telegram pada tahap selanjutnya karena seluruh data aplikasi tersimpan pada direktori tersebut.

7. PEMETAAN STRUKTUR FOLDER TELEGRAM

Setelah mendapatkan informasi path dari data telegramnya, selanjutnya kita akan melihat ada folder apa saja didalam **org.telegram.messenger.web**.

Disini kami tetap berada di shell lalu ketikkan perintah "**ls -la /data/data/org.telegram.messenger.web**". Nantinya akan menampilkan folder folder yang berada di dalamnya

```

generic_x86:/ # ls -la /data/data/org.telegram.messenger.web
total 96
drwx----- 10 u0_a146 u0_a146      4096 2026-06-23 15:12 .
drwxrwx--x 180 system system      8192 2026-06-22 18:21 ..
drwxrwx--x  2 u0_a146 u0_a146      4096 2026-06-22 18:22 app_textures
drwx-----  4 u0_a146 u0_a146      4096 2026-06-22 18:22 app_webview
drwxrws--x  5 u0_a146 u0_a146_cache 4096 2026-06-22 22:55 cache
drwxrws--x  2 u0_a146 u0_a146_cache 4096 2026-06-22 18:21 code_cache
drwxrwx--x  2 u0_a146 u0_a146      4096 2026-06-22 18:22 databases
drwxrwx--x  6 u0_a146 u0_a146      8192 2026-06-23 01:36 files
lrwxrwxrwx  1 root root              74 2026-06-23 15:12 lib -> /data/app/org.telegram.messenger.web-IoR3ZESpB8bU13-4P
AfGkQ=~/lib/x86
drwxrwx--x  2 u0_a146 u0_a146      4096 2026-06-22 18:21 no_backup
drwxrwx--x  2 u0_a146 u0_a146      4096 2026-06-23 01:36 shared_prefs

```

Nah disini terdapat beberapa folder yaitu :

app_textures/

app_webview/

cache/

code_cache/

databases/

files/

no_backup/

shared_prefs/

Kesimpulan

Berdasarkan hasil pemetaan, direktori databases, files, dan shared_prefs diidentifikasi sebagai lokasi yang paling berpotensi menyimpan artefak digital yang relevan dengan aktivitas pengguna Telegram. Oleh karena itu, ketiga direktori tersebut menjadi target utama pada tahap akuisisi dan analisis berikutnya.

8. ENUMERASASI DIREKTORI DATABASE

Setelah direktori database Telegram berhasil ditemukan, dilakukan proses enumerasi untuk mengidentifikasi file database yang

tersedia pada aplikasi. Kami menggunakan perintah "**ls -la /data/data/org.telegram.messenger.web/databases**"

```
drwxrwx--x  2 u0_a146 u0_a146  4096 2026-06-22 18:22 .
drwx----- 10 u0_a146 u0_a146  4096 2026-06-23 15:12 ..
-rw-rw----  1 u0_a146 u0_a146 16384 2026-06-22 18:24 cesdb
-rw-rw----  1 u0_a146 u0_a146    0 2026-06-22 18:24 cesdb-journal
-rw-rw----  1 u0_a146 u0_a146 57344 2026-06-22 18:22 com.google.android.datatransport.events
-rw-rw----  1 u0_a146 u0_a146    0 2026-06-22 18:22 com.google.android.datatransport.events-journal
generic_x86:/ # ls -la data/data/org.telegram.messenger.web/databases/cesdb
-rw-rw----  1 u0_a146 u0_a146 16384 2026-06-22 18:24 data/data/org.telegram.messenger.web/databases/cesdb
```

Database yang muncul disini hanya **cesdb** dan **com.google.android.datatransport.events**, padahal untuk telegram android biasanya kami berharap menemukan **cache.db** , **tgnet.dat** atau database dengan ukuran beberapa mb hingga puluhan mb

Kesimpulan

Hasil pemeriksaan menunjukkan bahwa direktori databases hanya berisi beberapa file berukuran relatif kecil, yaitu:

- cesdb
- com.google.android.datatransport.events

Tidak ditemukan database komunikasi yang secara eksplisit menunjukkan penyimpanan pesan, kontak, maupun aktivitas pengguna Telegram. Berdasarkan ukuran file yang relatif kecil, diduga file tersebut tidak merupakan database utama yang menyimpan artefak komunikasi pengguna.

Temuan ini mengindikasikan bahwa aplikasi Telegram yang digunakan kemungkinan menyimpan data pengguna pada lokasi lain seperti direktori files, cache, atau penyimpanan WebView. Oleh karena itu proses investigasi dilanjutkan dengan melakukan pemeriksaan terhadap direktori penyimpanan lainnya.

9. AKUISISI DATABASE TELEGRAM

Karena di folder Database kami tidak menemukan file .db yang dicari, maka kami melakukan eksplorasi di 2 folder yang kami duga menyimpan databasenya, yaitu :

app_webview/

files/

kami mulai dari folder app_webview terlebih dahulu, caranya sama yaitu dengan mengetikkan perintah "**ls -la data/data/org.telegram.messenger.web/app_webview**". Berikut adalah output yang dikeluarkan

```
generic_x86:/ # ls -la data/data/org.telegram.messenger.web/app_webview
total 124
drwx----- 4 u0_a146 u0_a146 4096 2026-06-22 18:22 .
drwx----- 10 u0_a146 u0_a146 4096 2026-06-23 15:12 ..
drwx----- 3 u0_a146 u0_a146 4096 2026-06-22 18:22 GPUCache
-rw----- 1 u0_a146 u0_a146 57344 2026-06-22 18:22 Web\ Data
-rw----- 1 u0_a146 u0_a146 0 2026-06-22 18:22 Web\ Data-journal
drwx----- 3 u0_a146 u0_a146 4096 2026-06-22 18:22 blob_storage
-rw----- 1 u0_a146 u0_a146 36 2026-06-22 18:22 metrics_guid
-rw----- 1 u0_a146 u0_a146 55 2026-06-22 18:22 pref_store
-rw----- 1 u0_a146 u0_a146 0 2026-06-22 18:22 variations_seed_new
-rw----- 1 u0_a146 u0_a146 0 2026-06-22 18:22 variations_stamp
-rw----- 1 u0_a146 u0_a146 0 2026-06-22 18:22 webview_data.lock
generic_x86:/ #
```

Karena telegram nya berbasis.web kami kira file databasenya disimpan di folder webview, ternyata di webview tidak ada.

Sekanjutnya kami akan menganalisis folder **files/** dengan perintah yang hampir sama dan hanya mengganti path folder di belakangnya saja menjadi **files/** "**ls -la data/data/org.telegram.messenger.web/files**". Lalu begini outputnya :

```

drwx----- 3 u0_a146 u0_a146 4096 2026-06-22 18:22 ShortcutInfoCompatSaver_share_targets
drwx----- 2 u0_a146 u0_a146 4096 2026-06-22 22:59 account1
drwx----- 2 u0_a146 u0_a146 4096 2026-06-22 22:59 account2
drwx----- 2 u0_a146 u0_a146 4096 2026-06-22 22:59 account3
-rw----- 1 u0_a146 u0_a146 8922 2026-06-22 18:22 arctic.attheme
-rw----- 1 u0_a146 u0_a146 5876 2026-06-22 18:21 bluebubbles.attheme
-rw----- 1 u0_a146 u0_a146 10862592 2026-06-22 23:51 cache4.db
-rw----- 1 u0_a146 u0_a146 32768 2026-06-23 15:13 cache4.db-shm
-rw----- 1 u0_a146 u0_a146 4552632 2026-06-23 01:36 cache4.db-wal
-rw----- 1 u0_a146 u0_a146 1990 2026-06-22 22:49 cthumb.jpg
-rw----- 1 u0_a146 u0_a146 15194 2026-06-22 18:22 darkblue.attheme
-rw----- 1 u0_a146 u0_a146 9663 2026-06-22 18:22 day.attheme
-rw-rw---- 1 u0_a146 u0_a146 40 2026-06-22 22:47 dc1conf.dat
-rw-rw---- 1 u0_a146 u0_a146 40 2026-06-22 22:56 dc2conf.dat
-rw-rw---- 1 u0_a146 u0_a146 40 2026-06-22 22:57 dc4conf.dat
-rw-rw---- 1 u0_a146 u0_a146 40 2026-06-23 01:36 dc5conf.dat
-rw----- 1 u0_a146 u0_a146 24576 2026-06-22 22:53 file_to_path.db
-rw----- 1 u0_a146 u0_a146 24576 2026-06-22 22:42 file_to_path_backup.db
-rw----- 1 u0_a146 u0_a146 0 2026-06-22 18:21 generatefid.lock
-rw----- 1 u0_a146 u0_a146 15882 2026-06-22 18:22 night.attheme
-rw----- 1 u0_a146 u0_a146 24 2026-06-23 00:04 profileInstalled
-rw----- 1 u0_a146 u0_a146 8 2026-06-22 18:21 profileinstaller_profileWrittenFor_lastUpdateTime.dat
-rw----- 1 u0_a146 u0_a146 1060739 2026-06-22 18:21 remote_en.xml
-rw----- 1 u0_a146 u0_a146 612 2026-06-23 01:37 stats2.dat
-rw-rw---- 1 u0_a146 u0_a146 5448 2026-06-23 01:20 tgnet.dat

```

Disini kami akhirnya menemukan artefak database yang kami cari yang bernama **cache4.db**, karena ukuran file tersebut tergolong besar daripada file lainnya yaitu sebesar 10 mb, disitu juga tertera **cache4.db-wal** dan **cache4.db-shm**. ini adalah database yang sangat khas di telegram.

Artefak Penting Yang Dikumpulkan

NO	File	Dugaan Fungsi
1.	cache4.db	Database utama Telegram
2.	cache4.db-wal	Write Ahead Log SQLite
3.	cache4.db-shm	Shared Memory SQLite
4.	tgnet.dat	Konfigurasi jaringan Telegram
5.	file_to_path.db	Mapping file/media
6.	account 1,2,3	Data akun

Langkah selanjutnya kita buat folder di dalam folder bernama Database di dalam folder Telegram yang sudah kita buat sebelumnya, ini berfungsi untuk menampung file database yang nantinya akan kita pull. Setelah selesai membuat, sekarang kita akan melakukan

adb pull pada beberapa file tersebut menggunakan perintah perintah dibawah ini:

cache4.db :

```
./adb pull  
/data/data/org.telegram.messenger.web/files/cache4.db "..\Logical_Acquisition\Telegram\Database\cache4.db"
```

cache4.db-wal :

```
./adb pull  
/data/data/org.telegram.messenger.web/files/cache4.db-wal "..\Logical_Acquisition\Telegram\Database\cache4.db-wal"
```

cache4.db-shm :

```
./adb pull  
/data/data/org.telegram.messenger.web/files/cache4.db-shm  
"..\Logical_Acquisition\Telegram\Database\cache4.db-shm"
```

tgnet.dat :

```
./adb pull  
/data/data/org.telegram.messenger.web/files/tgnet.dat "..\Logical_Acquisition\Telegram\Database\tgnet.dat"
```

```
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb pull /data/data/org.telegram.messenger.web/files/cache4.db "..\Logical_Acquisition\Telegram\Database\cache4.db"  
/data/data/org.telegram.messenger.web/files/cache4.db: 1 file pulled, 0 skipped. 68.8 MB/s (10862592 bytes in 0.151s)  
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb pull /data/data/org.telegram.messenger.web/files/cache4.db-wal "..\Logical_Acquisition\Telegram\Database\cache4.db-wal"  
/data/data/org.telegram.messenger.web/files/cache4.db-wal: 1 file pulled, 0 skipped. 65.5 MB/s (4552632 bytes in 0.066s)  
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb pull /data/data/org.telegram.messenger.web/files/cache4.db-shm "..\Logical_Acquisition\Telegram\Database\cache4.db-shm"  
/data/data/org.telegram.messenger.web/files/cache4.db-shm: 1 file pulled, 0 skipped. 8.1 MB/s (32768 bytes in 0.004s)  
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\ADB> ./adb pull /data/data/org.telegram.messenger.web/files/tgnet.dat "..\Logical_Acquisition\Telegram\Database\tgnet.dat"  
/data/data/org.telegram.messenger.web/files/tgnet.dat: 1 file pulled, 0 skipped. 1.5 MB/s (5448 bytes in 0.003s)
```

Name	Date modified	Type	Size
cache4.db	23/06/2026 17:35	Data Base File	10.608 KB
cache4.db-wal	23/06/2026 17:35	DB-WAL File	4.446 KB
cache4.db-shm	23/06/2026 17:35	DB-SHM File	32 KB
tgnet.dat	23/06/2026 17:38	DAT File	6 KB

Setelah semua file penting di pull, selanjutnya kita akan menghitung hash SHA256 dari database **cache4.db** menggunakan perintah **"Get-FileHash cache4.db -Algorithm SHA256"**

```
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\Logikal_Acquisition\Telegram\Database> Get-FileHash cache4.db -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	C05CFB871D824EE7009544D49FA43C2F2B1CF21280EC655A1A37C28E36BA1F23	D:\Kuliah\Semester_6\Forensik Digital\UAS\Logikal_Acquisition\Tel...

C05CFB871D824EE7009544D49FA43C2F2B1CF21280EC655A1A37C28E36BA1F23

Tujuan proses hashing adalah memastikan bahwa file yang dianalisis memiliki integritas yang sama dengan file hasil akuisisi dan tidak mengalami perubahan selama proses penyimpanan maupun analisis.

Berdasarkan hasil perhitungan hash, artefak cache4.db berhasil diverifikasi dan dinyatakan siap untuk dilakukan analisis forensik lebih lanjut.

Kesimpulan

Berdasarkan hasil pemetaan direktori aplikasi Telegram, ditemukan bahwa database utama aplikasi tidak disimpan pada direktori databases sebagaimana struktur aplikasi Android pada umumnya. Database utama Telegram ditemukan pada direktori:

/data/data/org.telegram.messenger.web/files/

Artefak yang berhasil diidentifikasi antara lain:

- cache4.db
- cache4.db-wal`
- cache4.db-shm
- tgnet.dat

File cache4.db memiliki ukuran sekitar 10,8 MB dan diidentifikasi sebagai kandidat utama database Telegram yang berpotensi menyimpan artefak komunikasi pengguna. Selain itu ditemukan file cache4.db-wal dan cache4.db-shm yang merupakan bagian dari mekanisme SQLite Write-Ahead Logging (WAL).

Seluruh file tersebut kemudian diakuisisi menggunakan Android Debug Bridge (ADB) dan disimpan pada workstation investigator untuk dilakukan proses analisis lebih lanjut.

10. PHYSICAL ACQUISITION

Setelah proses Logical Acquisition selesai dilakukan, investigator melanjutkan proses Physical Acquisition terhadap media penyimpanan emulator Android.

Pada lingkungan Android Emulator, data pengguna disimpan dalam image file virtual yang merepresentasikan partisi penyimpanan perangkat. Oleh karena itu proses Physical Acquisition dilakukan dengan mengakuisisi file image yang digunakan oleh Android Virtual Device (AVD).

Nah Lokasi image emulator di pc kami berada di path berikut ini :

C:\Users\Arka.android\avd\Pixel_2.avd

Name	Date modified	Type	Size
data	22/06/2026 18:20	File folder	
hardware-qemu.ini.lock	23/06/2026 15:12	File folder	
snapshots	23/06/2026 15:12	File folder	
tmpAdbCmds	23/06/2026 15:13	File folder	
AVD.conf	23/06/2026 15:12	CONF File	1 KB
bootcompleted.ini	23/06/2026 15:12	Configuration setti...	0 KB
cache.img	22/06/2026 18:14	Disc Image File	67.584 KB
cache.img.qcow2	23/06/2026 15:10	QCOW2 File	29.952 KB
config.ini	22/06/2026 18:18	Configuration setti...	2 KB
emu-launch-params.txt	23/06/2026 15:12	Text Document	1 KB
encryptionkey.img	22/06/2026 17:54	Disc Image File	1.024 KB
encryptionkey.img.qcow2	23/06/2026 15:10	QCOW2 File	321 KB
hardware-qemu.ini	23/06/2026 15:12	Configuration setti...	5 KB
multiinstance.lock	23/06/2026 15:12	LOCK File	0 KB
qemu-version.txt	22/06/2026 18:14	Text Document	1 KB
quickbootChoice.ini	23/06/2026 01:43	Configuration setti...	1 KB
read-snapshot.txt	23/06/2026 15:12	Text Document	0 KB
sdcard.img	22/06/2026 18:13	Disc Image File	524.288 KB
sdcard.img.qcow2	23/06/2026 15:10	QCOW2 File	1.024 KB
userdata.img	22/06/2026 17:54	Disc Image File	1.024 KB
userdata-qemu.img	22/06/2026 18:14	Disc Image File	6.291.456 ...
userdata-qemu.img.qcow2	23/06/2026 18:31	QCOW2 File	1.039.040 ...
user-settings.ini	22/06/2026 18:18	Configuration setti...	1 KB
version_num.cache	22/06/2026 18:14	CACHE File	1 KB

berikut adalah isi file atau folder yang tedapat di path tersebut, disini kita fokus kepada file **userdata-qemu.img** yang berukuran 1gb dan **userdata-qemu.img.qcow2** yang berukuran 6.2 gb. nantinya yang akan kita akuisisi adalah file file **userdata-qemu.img.qcow2** karena ukuranya lebih kecil, kemungkinan besar file ini yang berisi perubahan yang terjadi akibat instalasi Telegram, SMS, Call Log, media, aktivitas pengguna. Lalu kita akan masuk ke tahapan physical acquisition

Langkah 1 – Salin File Barang Bukti

Langkah pertama yaitu membuat folder bernama Physical_Acquisition yang nantinya akan menampung image nya, lalu salin **userdata-qemu.img** dan **userdata-qemu.img.qcow2** dari **C:\Users\Arka.android\avd\Pixel_2.avd** ke folder yang baru dibuat

Name	Date modified	Type	Size
 userdata-qemu.img	22/06/2026 18:14	Disc Image File	6.291.456 ...
 userdata-qemu.img.qcow2	23/06/2026 18:58	QCOW2 File	1.041.856 ...

Langkah 2 – Hitung Hash

Selanjutnya arahkan path powershell ke folder yang baru dibuat tersebut lalu jalankan perintah berikut ini :

userdata-qemu.img :

Get-FileHash userdata-qemu.img -Algorithm SHA256

userdata-qemu.img.qcow2 :

Get-FileHash userdata-qemu.img.qcow2 -Algorithm SHA256

```
PS D:\Kuliah\Semester_6\Forensik Digital\UAS\Physical_Acquisition> ls

Directory: D:\Kuliah\Semester_6\Forensik Digital\UAS\Physical_Acquisition

Mode                LastWriteTime         Length Name
----                -
-a----          6/22/2026   6:14 PM         6442458944 userdata-qemu.img
-a----          6/23/2026   6:58 PM        1066860544 userdata-qemu.img.qcow2

PS D:\Kuliah\Semester_6\Forensik Digital\UAS\Physical_Acquisition> Get-FileHash userdata-qemu.img -Algorithm SHA256

Algorithm      Hash
-----
SHA256         CC7FD869EA122B7C93689F0302C1EF086E052DC673D666F0BB809CA64E2D7DB3
Path
D:\Kuliah\Semester_6\Forensik Digital\UAS\Physical_Acquisition\us...

PS D:\Kuliah\Semester_6\Forensik Digital\UAS\Physical_Acquisition> Get-FileHash userdata-qemu.img.qcow2 -Algorithm SHA256

Algorithm      Hash
-----
SHA256         4824BB7C1FF876D79F924572FD4835536868D8D84D67015E38EB0CC13ED6C5DF
Path
D:\Kuliah\Semester_6\Forensik Digital\UAS\Physical_Acquisition\us...
```

Setelah mendapatkan hash nya,buat folder baru bernama hash, lalu masukkan hashnya ke format .txt dan simpan disana

Name	Date modified	Type	Size
 cache4hash.txt	23/06/2026 18:42	Text Document	1 KB
 userdata-qemu.img.qcow2.sha256.txt	23/06/2026 19:06	Text Document	1 KB
 userdata-qemu.img.sha256.txt.txt	23/06/2026 19:05	Text Document	1 KB

Kesimpulan

Kedua file tersebut merupakan representasi penyimpanan virtual perangkat Android yang digunakan selama simulasi aktivitas pengguna. File image kemudian diamankan dan dilakukan proses hashing menggunakan algoritma SHA256 untuk menjaga integritas barang bukti digital.

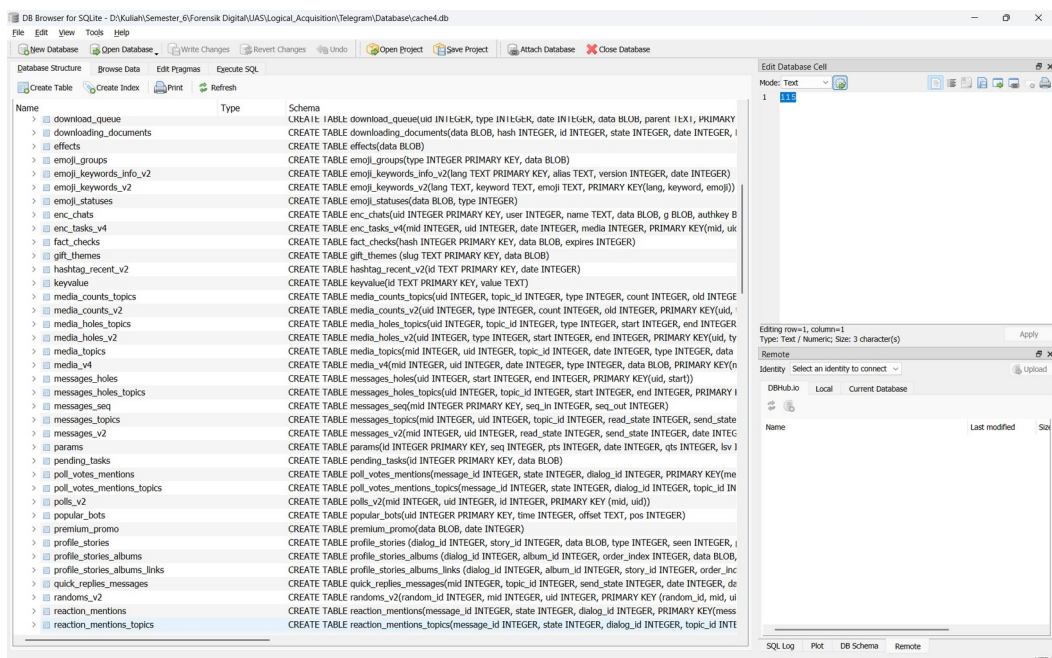
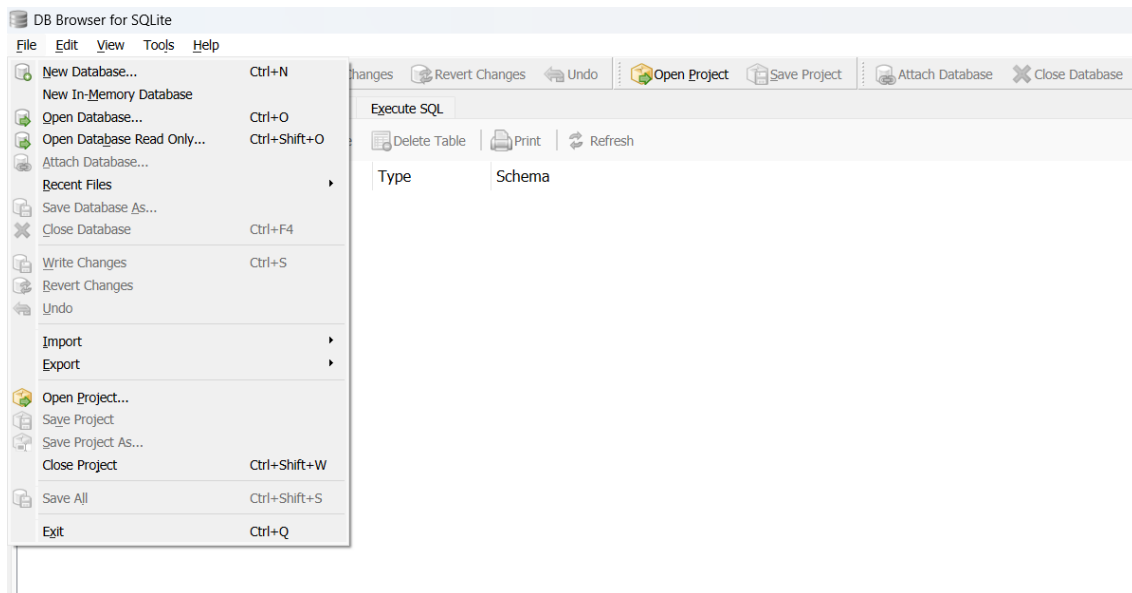
Hasil Physical Acquisition ini digunakan sebagai sumber bukti tingkat rendah (low-level evidence) yang dapat dianalisis lebih lanjut menggunakan perangkat lunak forensik apabila diperlukan.

11. ANALISIS DATABASE TELEGRAM

Setelah semua file siap, selanjutnya kami akan melakukan analisis database telegram yang tadi sudah didapatkan. Disini kami menggunakan aplikasi DB Browser (SQLite) dan yang akan menjadi target yaitu file **cache4.db**. Berikut adalah langkah langkahnya

Langkah 1 – Buka Database

Buka sqlite lalu pada menu file di pojok kiri atas pilih open database lalu pilih **cache4.db** yang tadi sudah di dapatkan



Nantinya akan terbuka banyak table yang tersedia

Langkah 2 – Lihat Struktur Tabel

Buka database structure lalu focus di nama nama table yang mengandung kata berikut ini :

message

user

chat

dialog

media

contact

Database Structure Browse Data Edit Pragmas Execute SQL																
Table: messages_v2 Filter in any column																
id	uid	read_state	send_state	date	data	out	ttl	media	replydata	imp	mention	forwards	replies_data	thread_reply_id	is_channel	reply_to_message_id
id	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
4	11	8840493868	2	0	1779959104	BLOB	0	0	0	NULL	0	0	0	NULL	0	0
5	28	8717431963	3	0	1773945460	BLOB	0	0	0	NULL	0	0	0	NULL	0	0
6	27	8761206944	2	0	1772868331	BLOB	0	0	0	NULL	0	0	0	NULL	0	0
7	26	8764420823	2	0	1772835077	BLOB	0	0	0	NULL	0	0	0	NULL	0	0
8	25	8716995619	2	0	1772378844	BLOB	0	0	0	NULL	0	0	0	NULL	0	0
9	17	1327557995	3	0	1782142859	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
10	19	1304847444	3	0	1782142923	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
11	22	1304847444	3	0	1782143022	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
12	24	1304847444	3	0	1782143094	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
13	25	1304847444	3	0	1782143131	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
14	32	1304847444	3	0	1782143353	BLOB	1	0	0	NULL	0	0	0	NULL	0	0
15	34	5340529263	3	0	1782143459	BLOB	1	0	0	NULL	0	0	0	NULL	0	0
16	33	5340529263	3	0	1782143403	BLOB	1	0	0	NULL	0	0	0	NULL	0	0
17	31	5340529263	3	0	1782143294	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
18	30	5340529263	3	0	1782143276	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
19	29	5340529263	3	0	1782143265	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
20	28	5340529263	3	0	1782143262	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
21	27	5340529263	3	0	1782143261	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
22	26	5340529263	3	0	1782143222	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
23	23	5340529263	3	0	1782143080	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
24	21	5340529263	3	0	1782142946	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
25	20	5340529263	3	0	1782142935	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
26	18	5340529263	3	0	1782142905	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
27	35	5340529263	3	0	1782143640	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
28	36	5340529263	3	0	1782143650	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
29	37	5340529263	3	0	1782143667	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
30	38	5340529263	3	0	1782143696	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0
31	39	5340529263	3	0	1782143742	BLOB	0	0	-1	NULL	0	0	0	NULL	0	0
32	40	1304847444	3	0	1782143772	BLOB	1	0	-1	NULL	0	0	0	NULL	0	0

tapi disini kami tidak bisa langsung membaca isi dari setiap tabelnya karena telegram menyimpan chat/grub/channel tidak dalam bentuk teks biasa.

Selanjutnya untuk analisis pergi ke bagian **execute sql** dan terapkan kode ini satu persatu untuk menemukan database yang bisa dibaca :

1.Perintah untuk melihat isi dari tabel chats

```
SELECT uid,name
```

```
FROM chats
```

```
LIMIT 30;
```

24	3962066024	rayan channel
25	5408404835	forensic grub

Disini kami menemukan nama dan uid grub yang tadi sudah kami buat yaitu Bernama forensic grub dengan uid = **"540840483"**

2.Perintah untuk melihat user siapa saja yang ada

```
SELECT uid,name
```

```
FROM users
```

```
LIMIT 30;
```

	uid	name
1	777000	telegram;;
2	3025529	haida md. noor;;haida
3	4477462	abu omar;;haidl
4	11266446	niraj kohli;;niraj
5	86396048	na;;ryan;
6	129542144	نيركب;;nirabot
7	140267078	tenor gif search;;gif
8	322759367	merabot;;merabot
9	592244098	pak fandi rahman;;
10	614575866	pak bogie;;
11	821886177	mas udit;;xchernobogx
12	974074628	aci;;hemmo199610
13	988877678	claura;;heavenlyden
14	994847966	satyamahinsa asprak;;satyamahinsaa
15	1044142446	nisak;;haihainisak
16	1063642834	rafi;;aaraaip
17	1065701361	tasya;;tasyanau
18	1082146689	davin;;dapinygy
19	1103976244	asti;;skiessieiei
20	1116027340	gibrontol;;
21	1145014304	mas feri;;sam16758
22	1152644268	ogik;;luvsana10
23	1164498505	haikal new;;makalewjunior
24	1191863821	nasywa;;nasywaduh
25	1194650764	setyo (2024);;settzgrant
26	1204604209	riskha;;casseyiy
27	1222610710	shahid khalid;;

Nah karena telegram kami disini adalah telegram asli,jadinya banyak nomor yang ikut ke capture, daripada satu satu mencari kami menggunakan metode yang instan, yaitu dengan pergi ke bagian browse data lalu pilih table user setelah itu sorting menggunakan nama. Disini kami bisa menemukan 3 nama dan uid user yang sebelumnya sudah menjalankan tes Bersama kami

User 1 haidar :

New Database Open Database Write Changes Revert Changes Undo				
Database Structure Browse Data Edit Pragmas Execute SQL				
Table: users				
	uid	name ▲	status	data
	Filter	haidar	Filter	Filter
1	5340529263	haidar;;;hedarroi	-1000	BLOB

Uid dari Haidar yaitu **"5340529263"**

User 2 ryan :

2	1304847444	ryan.;;;ginzzen	-1000	BLOB
---	------------	-----------------	-------	------

Uid dari ryan yaitu **"1304847444"**

User 3 meraaa :

6	1327557995	meraa;;;meracya	-100	BLOB
---	------------	-----------------	------	------

Uid dari meraa yaitu **"1327557995"**

Kesimpulan

Analisis terhadap database cache4.db menunjukkan bahwa Telegram menyimpan artefak digital pada beberapa tabel utama, yaitu users, chats, dialogs, messages_v2, dan media_v2.

Struktur tabel users terdiri atas atribut uid, name, status, dan data. Tabel ini digunakan untuk menyimpan informasi pengguna atau kontak yang dikenali oleh aplikasi Telegram.

Struktur tabel chats terdiri atas atribut uid, name, dan data. Tabel ini digunakan untuk menyimpan informasi grup, channel, maupun percakapan yang terdapat pada akun Telegram pengguna.

Tabel `messages_v2` merupakan tabel utama yang menyimpan artefak pesan. Berdasarkan hasil analisis struktur tabel, setiap pesan memiliki atribut Message ID (`mid`), Dialog ID (`uid`), Timestamp (`date`), Status Pesan Keluar/Masuk (`out`), serta Data Pesan (`data`).

Hasil analisis menunjukkan bahwa Telegram tidak menyimpan isi pesan dalam bentuk teks biasa, melainkan dalam format Binary Large Object (BLOB). Oleh karena itu diperlukan proses decoding terhadap data BLOB untuk memperoleh isi pesan yang dapat dibaca manusia.

12. DECODING ARTEFAK TELEGRAM MENGUNAKAN PYTHON

Berdasarkan hasil analisis struktur database Telegram, diketahui bahwa sebagian besar informasi pesan disimpan pada tabel `messages_v2` dalam atribut `data` yang bertipe Binary Large Object (BLOB). Format penyimpanan tersebut menyebabkan isi pesan tidak dapat dibaca secara langsung menggunakan DB Browser for SQLite.

Untuk memperoleh artefak yang dapat dibaca manusia (human-readable), dilakukan proses decoding terhadap data BLOB menggunakan bahasa pemrograman Python. Metode ini dilakukan dengan membaca database `cache4.db`, mengekstraksi record dari tabel `messages_v2`, kemudian menghubungkan setiap pesan dengan identitas pengguna yang tersimpan pada tabel `users` dan `chats`.

Berikut adalah cuplikan kode yang kami gunakan :

```
decode_telegram.py X
decode_telegram.py > ...
1  import sqlite3
2  from datetime import datetime, timezone, timedelta
3  from itertools import groupby
4  import re
5
6  WIB = timezone(timedelta(hours=7))
7
8  conn = sqlite3.connect(r'D:\Kuliah\Semester_6\Forensik Digital\UAS\Logical_Acquisition\Telegram\Database\cache4.db')
9  cursor = conn.cursor()
10
11 TARGET_UIDS = [1304847444, 5340529263, 1327557995, 540840483]
12
```

Disini bagian pentingnya berada di target_uids, di target ini kami harus sesuaikan dengan uid yang sudah kami dapatkan tadi, itulah mengapa kami mencatat uid setiap orang yang tergabung ke skenario

Proses decoding dilakukan dengan tahapan sebagai berikut:

1. Membuka database cache4.db menggunakan library sqlite3.
2. Mengambil data pengguna dari tabel users.
3. Mengambil data percakapan dari tabel messages_v2.
4. Menghubungkan UID pengguna dengan pesan yang terkait.
5. Mengekstraksi informasi yang masih dapat dibaca dari data BLOB menggunakan proses parsing dan filtering karakter.
6. Mengonversi timestamp Unix menjadi format waktu lokal (WIB).
7. Menyusun hasil ekstraksi menjadi bentuk timeline komunikasi.

Selain melakukan decoding pesan, script Python juga digunakan untuk melakukan deteksi kemungkinan pesan yang telah dihapus. Deteksi dilakukan dengan menganalisis urutan Message ID (MID) pada tabel messages_v2. Apabila ditemukan loncatan (gap) pada urutan MID, maka kondisi tersebut dapat mengindikasikan adanya pesan yang tidak lagi tersedia dalam database.

Hasil decoding digunakan sebagai dasar dalam proses rekonstruksi timeline aktivitas pengguna yang meliputi pengiriman pesan,

penerimaan pesan, pertukaran media, komunikasi grup, serta aktivitas lain yang berhasil ditemukan pada artefak Telegram

13. HASIL DECODING ARTEFAK TELEGRAM

1. Pesan di kirim dan di Terima

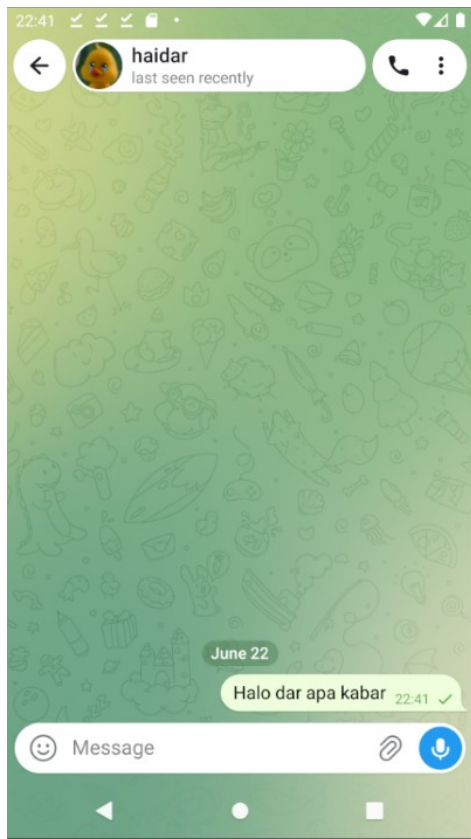
```
[2026-06-22 22:41:45 WIB] >>> KIRIM | haidar (uid=5340529263)  
Pesan: QYo Halo dar apa kabar
```

```
[2026-06-22 22:42:15 WIB] <<< TERIMA | haidar (uid=5340529263)  
Pesan: QYo oi co
```

```
[2026-06-22 22:42:26 WIB] <<< TERIMA | haidar (uid=5340529263)  
Pesan: QYo W9j  
        kapan nyoki
```

Disini bisa dilihat bahwa pada tanggal 22-06-2022 pada pukul 22:41:4 wib User A (emulator) mengirim pesan kepada Haidar yang berbunyi "Halo dar apa kabar".

Lalu pada jam 22:42:15 haidar membalas

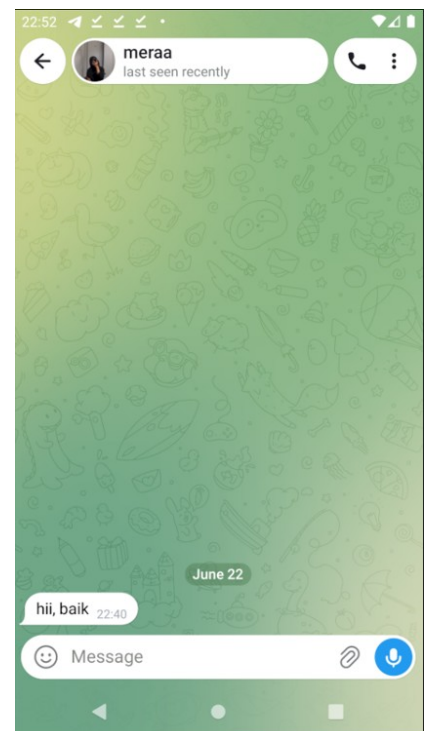


Hasil ini valid jika kita melihat ss an langsung dari telegram nya.baik secara tanggal dan waktunya

2. Pesan di kirim hapus

```
[2026-06-22 22:40:59 WIB] <<< TERIMA | meraa (uid=1327557995)
Pesan: QYk O QYk O W9j          hii, baik
```

Nah disini agak berbeda, karena di output python hanya ada balasan dari user bernama meraa yang berisi "hii, baik"



Padahal jika kami lihat di telegram nya, sebelum user meraa membalas pesan hii, baik, User A(emulator) terlebih dahulu mengirimkan pesan yang berbunyi “halo kak nira apa kabar”. Sebelum User A menghapus pesan nya, ini membuktikan bahwa pesan yang sudah dihapus tidak bisa dilihat isi dari pesan nya

3. Mengirim emoji, foto dan video

```
[2026-06-22 22:47:41 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: QYo 🥰🥰

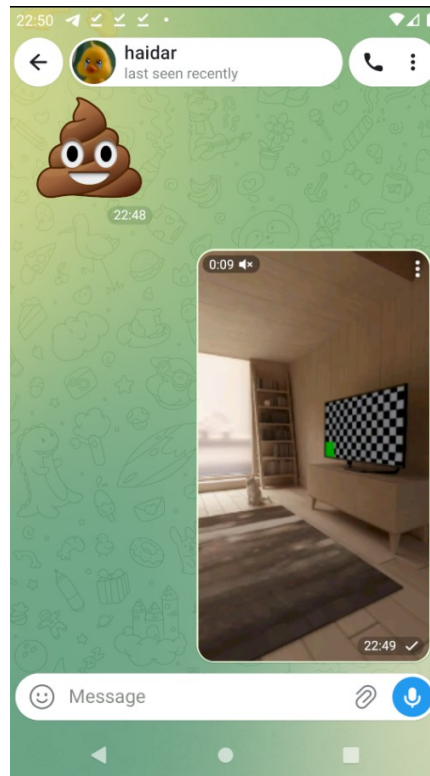
[2026-06-22 22:47:42 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: QYo 💋💋

[2026-06-22 22:47:45 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: QYo !Y9j ❤️❤️
```

Disini bisa dilihat bahwa emoji,foto dan video juga bisa dicapture saat melakukan akuisisi

```
[2026-06-22 22:49:13 WIB] >>> KIRIM | ryan. (uid=1304847444)
Pesan: [📷 FOTO]

[2026-06-22 22:50:03 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: [📺 VIDEO]
```



4. Voicecall

```
[2026-06-22 22:54:27 WIB] <<< TERIMA | haidar (uid=5340529263)
Pesan: [📞 PHONE CALL]

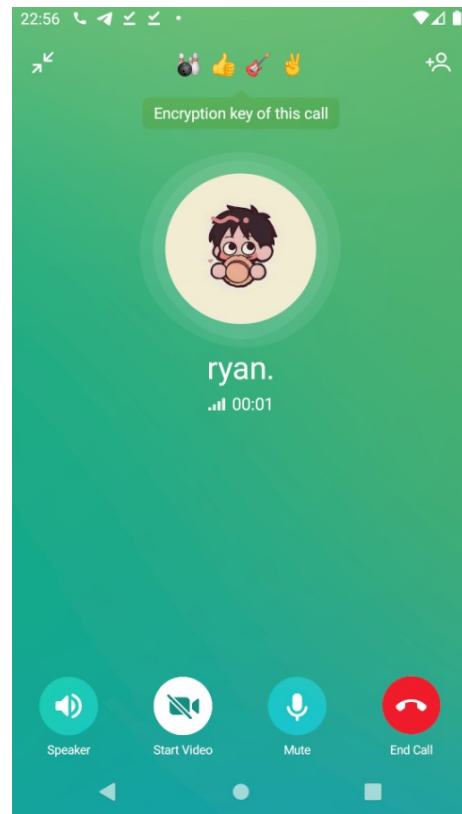
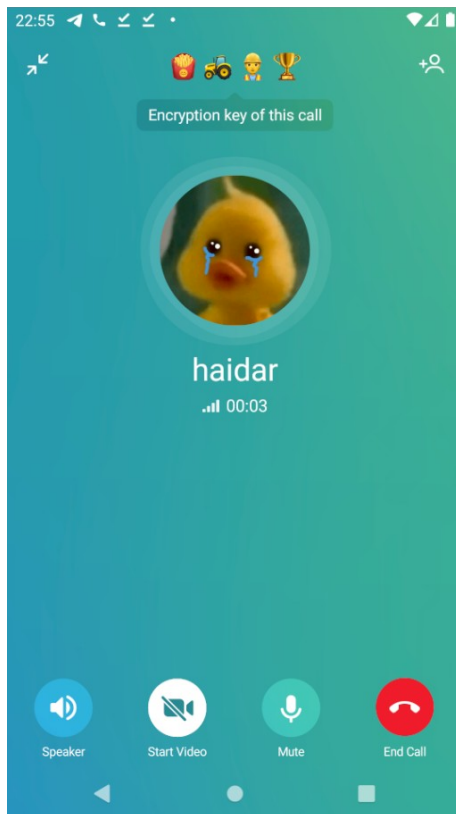
[2026-06-22 22:54:56 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: [📞 PHONE CALL]

[2026-06-22 22:55:42 WIB] <<< TERIMA | haidar (uid=5340529263)
Pesan: [📞 PHONE CALL]

[2026-06-22 22:56:12 WIB] >>> KIRIM | ryan. (uid=1304847444)
Pesan: [📞 PHONE CALL]

[2026-06-22 22:57:08 WIB] >>> KIRIM | ryan. (uid=1304847444)
Pesan: [📞 PHONE CALL]
```

Disini terdapat informasi mengenai voice call mulai dari telepon masuk dan keluar



5. Pesan yang dihapus

selain melakukan rekonstruksi komunikasi, script Python juga digunakan untuk melakukan analisis terhadap kemungkinan pesan yang telah dihapus.

Metode yang digunakan adalah dengan memeriksa urutan Message ID (MID) pada tabel messages_v2. Pada kondisi normal, MID akan bertambah secara berurutan. Apabila ditemukan loncatan nilai MID (gap), maka kondisi tersebut dapat mengindikasikan adanya record pesan yang tidak lagi tersedia dalam database.

Hasil analisis menunjukkan beberapa gap pada urutan MID, antara lain:

- MID 120-121
- MID 123
- MID 126-131

- MID 133-139

Temuan tersebut tidak dapat secara langsung membuktikan bahwa pesan telah dihapus, namun dapat digunakan sebagai indikator adanya artefak komunikasi yang tidak lagi tersedia pada database Telegram saat proses akuisisi dilakukan.

Oleh karena itu temuan tersebut dikategorikan sebagai suspected deleted messages atau dugaan pesan terhapus.

```
=====
DETEKSI PESAN YANG DIHAPUS
=====

[!] ryan. (uid=1304847444)
  ⚠ mid 120 s/d 121 HILANG (2 pesan kemungkinan dihapus)
  ⚠ mid 123 s/d 123 HILANG (1 pesan kemungkinan dihapus)
  ⚠ mid 126 s/d 131 HILANG (6 pesan kemungkinan dihapus)
  ⚠ mid 133 s/d 139 HILANG (7 pesan kemungkinan dihapus)

[!] haider (uid=5340529263)
  ⚠ mid 119 s/d 119 HILANG (1 pesan kemungkinan dihapus)
  ⚠ mid 122 s/d 122 HILANG (1 pesan kemungkinan dihapus)
  ⚠ mid 124 s/d 125 HILANG (2 pesan kemungkinan dihapus)
  ⚠ mid 132 s/d 132 HILANG (1 pesan kemungkinan dihapus)
```

6. Edit pesan



Jadi disini User A melakukan edit pesan ke haidar, tapi di output nya tidak ditemukan pesan sebelumnya jadi pesan sebelumnya langsung ditimpa oleh pesan yang baru

```
[2026-06-22 22:50:03 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: [ 📺 VIDEO]

[2026-06-22 22:50:59 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: QYo INI tadi aku ke pantai <Z9j

[2026-06-22 22:54:00 WIB] >>> KIRIM | haidar (uid=5340529263)
Pesan: [ 📞 PHONE CALL]
```

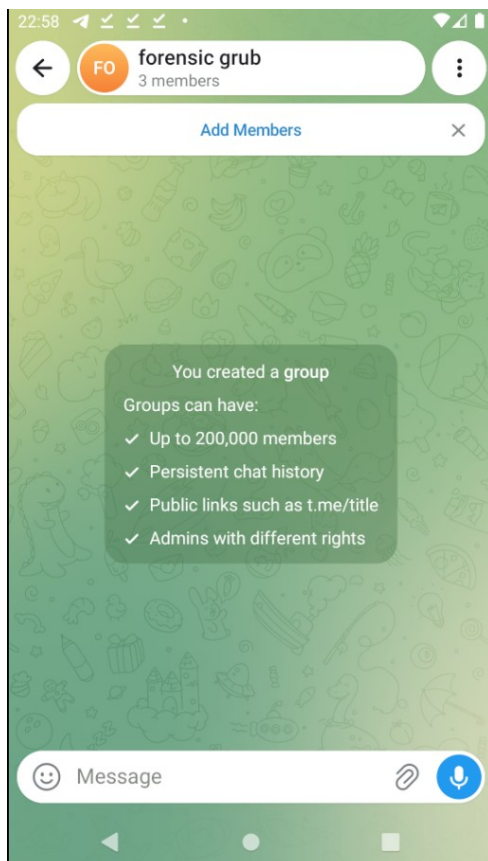
Analisis terhadap artefak Telegram menunjukkan bahwa pesan hasil edit berhasil ditemukan pada database. Namun investigator tidak menemukan versi pesan sebelum proses pengeditan dilakukan. Temuan ini mengindikasikan bahwa Telegram menyimpan versi pesan yang telah diperbarui (updated message) dan tidak mempertahankan salinan pesan sebelum proses edit pada artefak yang berhasil diekstraksi.

7.Membuat grub

Pada tahap simulasi aktivitas pengguna, investigator membuat sebuah grup Telegram bernama "forensic grub". Keberadaan grup

tersebut berhasil ditemukan pada tabel chats di dalam database Telegram.

Namun setelah dilakukan pencarian terhadap tabel messages_v2 menggunakan UID grup yang ditemukan, tidak ditemukan record pesan yang berasosiasi dengan UID tersebut.



25 | 5408404835 | forensic grub

Pada tahap simulasi aktivitas pengguna, investigator membuat sebuah grup Telegram bernama "forensic grub". Keberadaan grup tersebut berhasil ditemukan pada tabel chats di dalam database Telegram.

Namun setelah dilakukan pencarian terhadap tabel messages_v2 menggunakan UID grup yang ditemukan, tidak ditemukan record pesan yang berasosiasi dengan UID tersebut.

Hasil ini menunjukkan bahwa meskipun metadata grup berhasil ditemukan pada database Telegram, artefak komunikasi grup tidak

berhasil direkonstruksi menggunakan metode analisis yang digunakan pada penelitian ini.

Kemungkinan penyebab kondisi tersebut adalah perbedaan mekanisme penyimpanan pesan grup pada Telegram, penggunaan struktur data yang berbeda untuk grup dan channel, atau keterbatasan proses decoding terhadap artefak Telegram yang tersimpan dalam format BLOB.

Dengan demikian, investigasi berhasil membuktikan keberadaan grup, namun belum berhasil merekonstruksi isi komunikasi yang terjadi di dalam grup tersebut.